

20강

인터페이스

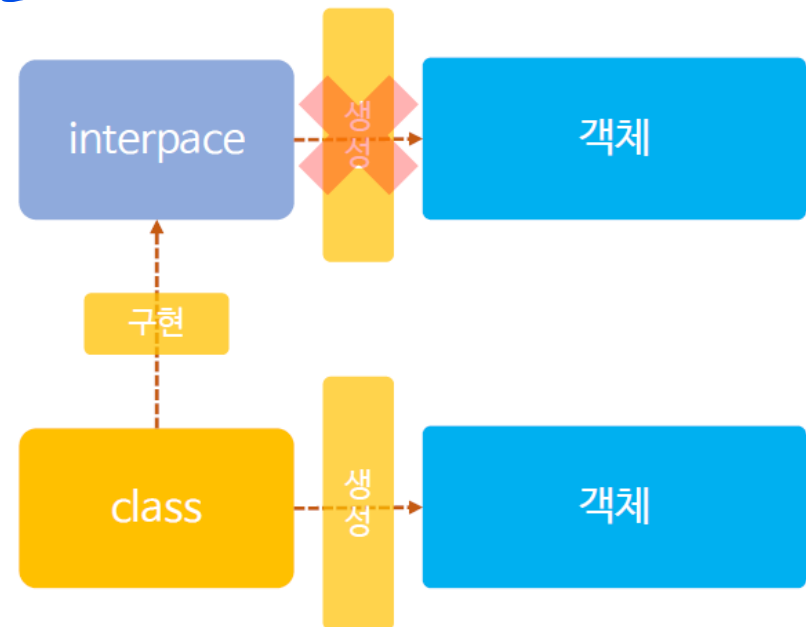
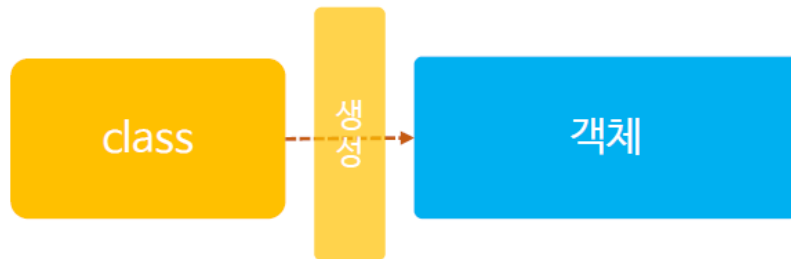




➤ 인터페이스(interface) 란?

- 클래스와 달리 객체를 생성(인스턴스) 할 수는 없으며, 클래스에서 구현해야 하는 작업 명세서이다.
- 인터페이스는 몸체가 없는 메서드들의 모임이다.
- 클래스를 구현하기 전에 만들 메서드를 먼저 정해놓는 것이다. (설계도와 같은 역할을 한다)
- 인터페이스(interface)는 오로지 추상 메소드와 상수만을 포함할 수 있다.

Java 7까지만





- 인터페이스를 사용하는 이유는 많지만, 가장 큰 이유는 객체가 다양한 자료형(타입)을 가질 수 있기 때문이다.
- 인터페이스를 이용하여 클래스를 구현하면 다른 클래스와 대체가 유연해서 유지보수가 편해진다는 장점이 있다. 마치 규격만 맞으면 다른 부품으로 바꿀 수 있는 것처럼 쉽게 변경 할 수 있기 때문입니다.
- 인터페이스는 잘 사용하면 프레임워크나 배포되는 공용 컴포넌트, 유틸 등을 개발할 때 아주 효과적이에요. 다만, 전제는 잘 사용해야 된다는 점이다. ✓
- 반대로, 일반 웹 개발 프로젝트에서는 오히려 복잡도만 증대시키는 상황이 많다. 많은 사람들이 불편해하고 필요성을 못 느낀다면 과감히 버리는 게 맞다.

```
접근제어자 interface 인터페이스이름 {  
    public static final 타입 상수이름 = 값;  
    ...  
    public abstract 메소드이름(매개변수목록);  
    ...  
}
```



➤ 인터페이스(interface) 란?

- 모든 메서드가 **추상 메서드(abstract method)**로 이루어진 클래스
- 형식적인 선언만 있고 구현은 없다.

```
Interface 인터페이스 이름{  
    public static final float pi = 3.14f;  
    public abstract void add();  
}
```

- 인터페이스에 선언된 모든 메서드는 **public abstract** 로 추상 메서드 이다.
- 인터페이스에 선언된 모든 변수는 **public static final**로 상수 이다.



➤ 클래스에서 인터페이스 구현하기

Calc 인터페이스를

Calculator 클래스에서 구현

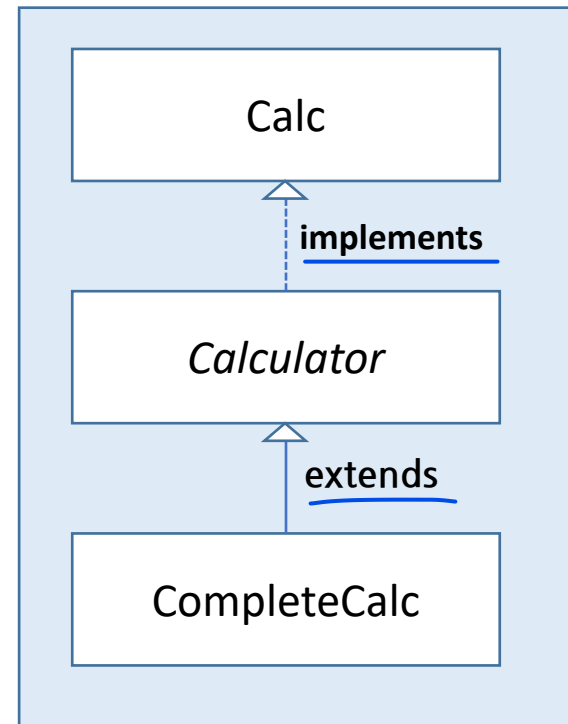
Calc의 모든 추상 메서드를 구현하지 않으면

Calculator는 추상 클래스가 됨

CompleteCalc 클래스가 **Calculator**를

상속받은 후 모든 메서드를 구현

CompleteCalc는 생성가능한 클래스





➤ Calc를 interface로 생성

```
public interface Calc {  
    // java 7이전에는 구현 코드가 있는 메소드가 존재하지 않았다.  
    // java 8이후 부터 구현 코드가 있는 메소드가 존재한다.  
    // 클래스 생성시 힙에어리어(Heap Area)에 메모리가 잡히는 것이 멤버변수이다.  
    // 인터페이스에서 멤버 변수는 public static final인 상수로 지정된다.
```

```
    public static final double PI = 3.14;  
    int ERROR = -999999999;
```

인터페이스에서 선언한 변수는 컴파일 과정에서 상수로 변환됨

```
    // interface 키워드 안에 작성이되면 무조건 abstract 메서드이다.  
    // 컴파일 될 때 public abstract 키워드가 들어 간다.
```

```
    public abstract int add(int num1, int num2);  
    int subtract(int num1, int num2);  
    int times(int num1, int num2);  
    int divide(int num1, int num2);
```

인터페이스에서 선언한 메서드는 컴파일 과정에서 추상 메서드로 변환됨

```
    // 그러므로 인터페이스에는 상수와 추상메서드가 선언된다.  
    // 인터페이스는 인스턴스화 할 수 없다.  
}
```



- Calculator는 Calc interface를 implements(상속)받는다.

```
public abstract class Calculator implements Calc {
```

```
// implements 는 인터페이스 타입을 상속했다. 라고 표현한다.
```

```
@Override
```

```
public int add(int num1, int num2) {  
    return num1 + num2;  
}
```

```
@Override
```

```
public int subtract(int num1, int num2) {  
    return num1 - num2;  
}
```

- CompleteCalc는 Calculator를 extends(상속)받는다.

```
public class CompleteCalc extends Calculator {
```

```
// extends는 구현 상속이라고 한다.
```

```
@Override
```

```
public int times(int num1, int num2) {  
    return num1 * num2;  
}
```

```
@Override
```

```
public int divide(int num1, int num2) {  
    if(num2 != 0) {  
        return num1 / num2;  
    }  
    return ERROR;  
}
```

```
public void showInfo() {
```

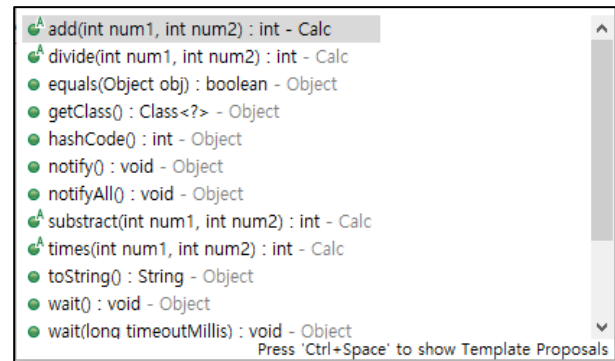
```
    System.out.println("Calc 인터페이스를 구현하겠습니다.");  
}
```



- 인터페이스를 구현한 클래스는 인터페이스 형으로 선언한 변수로 형 변환 할 수 있음
- 상속에서의 형 변환과 동일한 의미이다.
- 단 클래스 상속과 달리 구현코드가 없기 때문에 여러 인터페이스를 구현할 수 있음
- 형 변환시 사용할 수 있는 메서드는 인터페이스에 선언된 메서드만 사용할 수 있음

```
public class CalculatorTest {  
  
    public static void main(String[] args) {  
  
        int num1 = 10;  
        int num2 = 2;  
  
        Calc calc = new CompleteCalc();  
        /* 아래 처럼 인스턴스하는 것도 가능하다.  
        Calculator calc2 = new CompleteCalc();  
        CompleteCalc calc3 = new CompleteCalc();  
        */  
        // Calc calc1 = new Calc(); 인터페이스는 인스턴스 할 수 없다.  
        System.out.println(calc.add(num1, num2));  
    }  
}
```

CompleteCalc 가 Calc 형으로 대입된 경우
사용 할 수 있는 메서드





Calc **calc** = **new** CompleteCalc();

- **CompleteCalc** 가 **Calc** 형 으로 대입된 경우 사용할 수 있는 메서드
- 위의 코드를 일명 **Client code**라고 함
- **Client code** : 인터페이스를 기반으로 구현된 인스턴스를 가져다 사용한 다는 의미이다.



```
add(int num1, int num2) : int - Calc
divide(int num1, int num2) : int - Calc
equals(Object obj) : boolean - Object
getClass() : Class<?> - Object
hashCode() : int - Object
notify() : void - Object
notifyAll() : void - Object
subtract(int num1, int num2) : int - Calc
times(int num1, int num2) : int - Calc
toString() : String - Object
wait() : void - Object
wait(long timeoutMillis) : void - Object
Press 'Ctrl+Space' to show Template Proposals
```

Calc형 변수와 **Calculator**형 변수에서
사용 가능



add()

subtract()

times()

divide()

showInfo()



CompleteCalc 형 변수에서
사용 가능

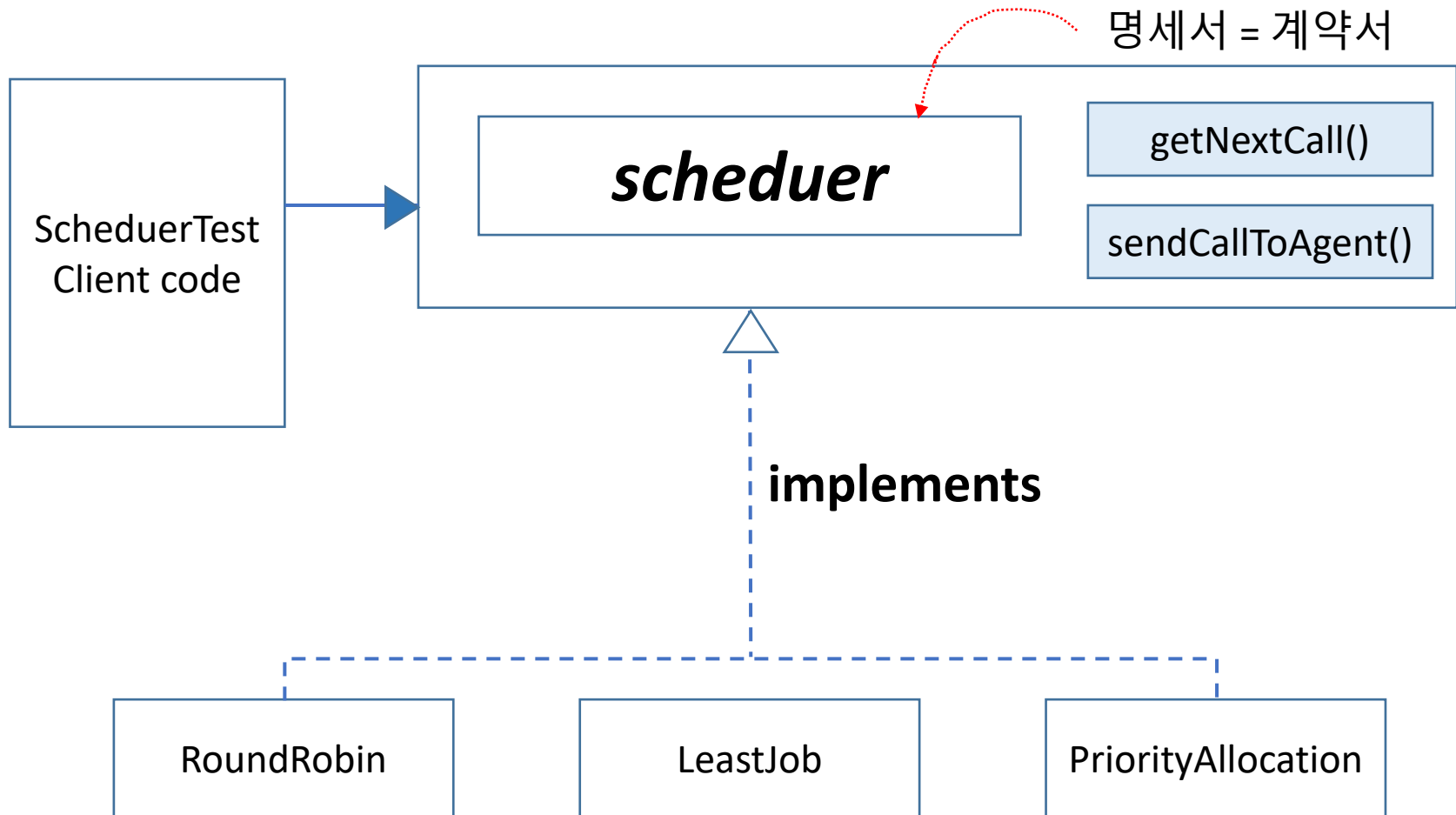


➤ 인터페이스와 다형성

- 인터페이스는 "Client Code"와 서비스를 제공하는 "객체" 사이의 약속이다
- 어떤 객체가 어떤 interface 타입이라 함은 그 interface가 제공하는 메서드를 구현했다는 의미이다.
- Client는 어떻게 구현되었는지 상관없이 interface의 정의만을 보고 사용 할 수 있다.(ex : JDBC)
- 다양한 구현이 필요한 인터페이스를 설계하는 일은 매우 중요한 일이다.
- 어떻게 사용하면 되는지를 interface가 알려준다.



➤ 인터페이스의 다형성 구현 구조





- Scheduler 인터페이스로 작성한다.

```
package scheduler;

public interface scheduler {

    void getNextCall(); //다음 고객의 call을 가져오는 메서드.
    void sendCallToAgent(); //상담원에게 call을 던지는 메서드
}
```

- RoundRobin 클래스는 scheduler 인터페이스를 implements 하여 작성한다.

```
public class RoundRobin implements scheduler{

    @Override
    public void getNextCall() {
        System.out.println("상담전화를 순서대로 대기열에서 가져옵니다.");
    }

    @Override
    public void sendCallToAgent() {
        System.out.println("다음 순서 상담원에게 배분합니다.");
    }
}
```



- LeastJob 클래스는 scheduler 인터페이스를 implements 하여 작성한다.

```
public class LeastJob implements scheduler{

    @Override
    public void getNextCall() {
        System.out.println("상담전화를 순서대로 대기열에서 가져옵니다.");
    }

    @Override
    public void sendCallToAgent() {
        System.out.println("현재 상담업무가 없거나 상담대기가 가장 적은 상담원에게 할당합니다.");
    }
}
```

- PriorityAllocation 클래스는 scheduler 인터페이스를 implements 하여 작성한다.

```
public class PriorityAllocation implements scheduler{

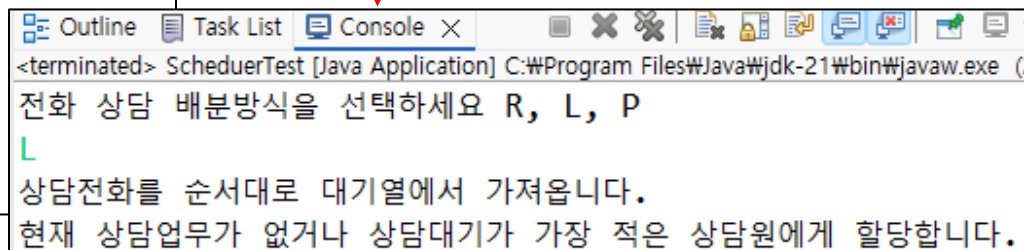
    @Override
    public void getNextCall() {
        System.out.println("고객의 등급이 높은 고객의 전화를 먼저 가져옵니다.");
    }

    @Override
    public void sendCallToAgent() {
        System.out.println("업무 skill 이 가장 높은 상담원의 대기열에 앞에 우선 배분합니다.");
    }
}
```



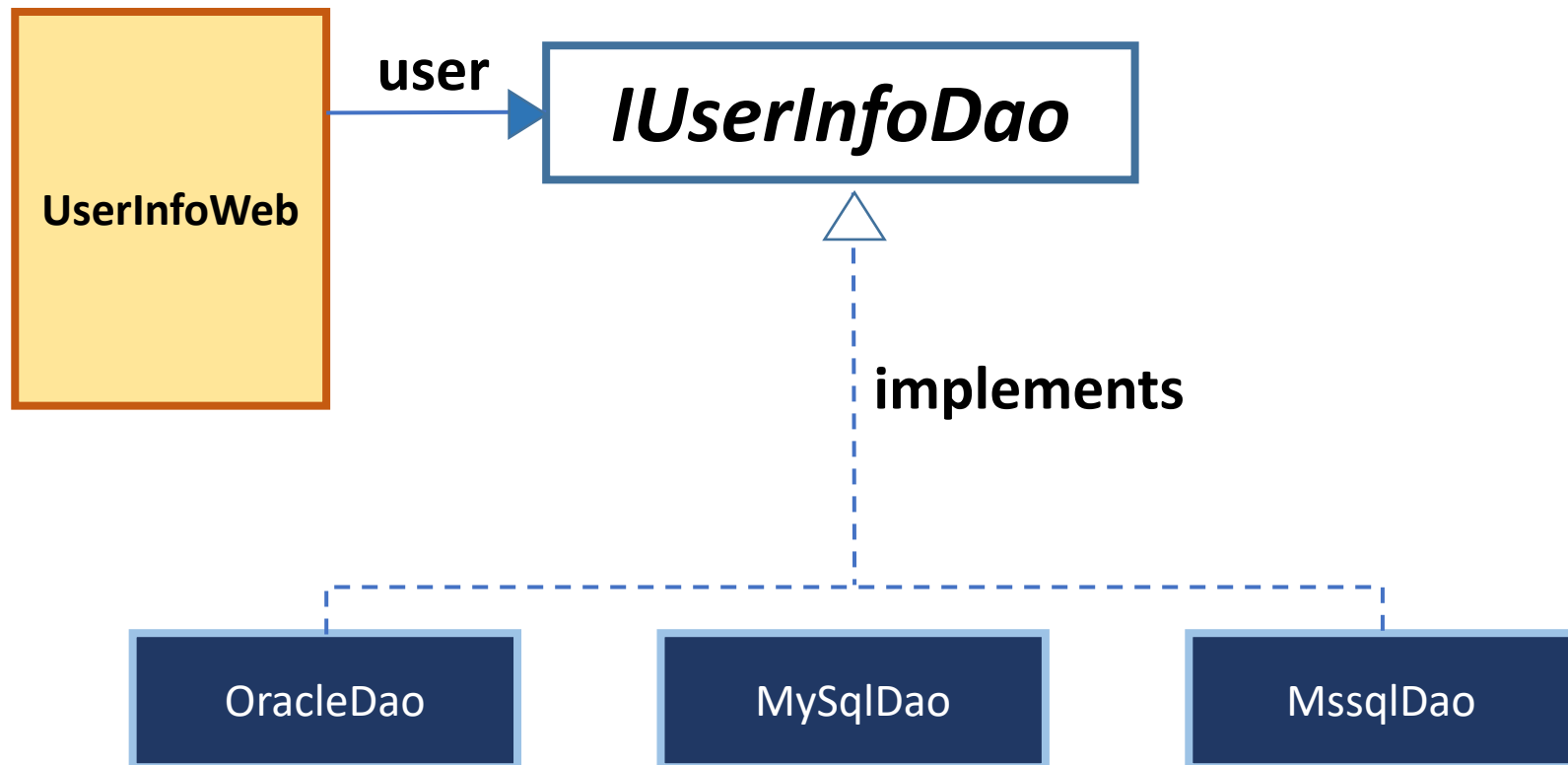
➤ 작성한 ScheduerTest 클래스는 Client code 이다.

```
public static void main(String[] args) {  
    // R, L, P  
    Scanner scan = new Scanner(System.in);  
    // RoundRobin 이 들어오면 R로,  
    // LeastJob 이 들어오면 L로  
    // PrionrityAllocation 이 들어오면 P로  
  
    System.out.println("전화 상담 배분방식을 선택하세요 R, L, P");  
    String ch = scan.next();  
    Scheduer sch = null;  
  
    if ("R".equals(ch) || "r".equals(ch)) {  
        sch = new RoundRobin();  
    } else if ("L".equals(ch) || "l".equals(ch)) {  
        sch = new LeastJob();  
    } else if ("P".equals(ch) || "p".equals(ch)) {  
        sch = new PriorityAllocation();  
    } else {  
        System.out.println("지원하지 않는 기능 입니다.");  
        return;  
    }  
  
    sch.getNextCall();  
    sch.sendCallToAgent();  
}
```





- **UserInfoWeb** 은 **IUserInfoDao** 에 정의된 메소드 명세만 보고 **Dao**를 사용할 수 있고, **Dao** 클래스들은 **IUserInfoDao** 에 정의된 메소드를 구현할 책임이 있다.





➤ 인터페이스의 요소

- 상수 : 모든 변수는 상수로 변환 됨
- 추상 메서드 : 모든 메서드는 추상 메서드로 구현
- Java version 8부터 아래 기능들이 추가 되었다.
 - **디폴트 메서드** : 기본 구현을 가지는 메서드, 구현 클래스에서 재정의 할 수 있음
 - **정적 메서드** : 인스턴스 생성과 상관 없이 인터페이스 타입으로 사용할 수 있는 메서드
 - **Private 메서드** : 인터페이스를 구현한 클래스에서 사용하거나 재정의 할 수 없음

default

인터페이스 내부에서만 기능을 제공하기 위해 구현하는 메서드



```
public interface Calc {
```

```
// 디폴트 메서드 : 기본 구현을 가지는 메서드로, 구현코드에서 재정의 할 수 있음
```

```
default void description() {  
    System.out.println("점수 계산기를 구현 합니다.");  
    myMethod();  
}
```

```
// 정적 메서드 : 인스턴스 생성과 상관없이 인터페이스 타입으로 사용할 수 있음
```

```
static int total(int[] arr) {  
    int total = 0;  
    for(int i=0;i<=arr.length;i++) {  
        total += i;  
    }  
}
```

```
myStaticMethod();  
return total;
```

```
}
```

```
// private 메서드 : 인터페이스 내부에서만 사용 가능, 구현한 클래스에서 사용하거나 재정의 할 수 없음
```

```
// private 메서드는 default 메서드에서 사용 가능
```

```
private void myMethod() {  
    System.out.println("private 메서드 입니다.");  
}
```

```
// private static 메서드는 static 메서드에서 사용 가능
```

```
private static void myStaticMethod() {  
    System.out.println("myStaticMethod 메서드 입니다.");  
}
```



```
package interfaceTest;

public class CaccluatorTest {

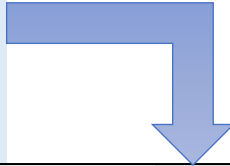
    public static void main(String[] args) {

        int num1 = 10;
        int num2 = 2;

        Calc calc = new CompleteCalc();
        calc.description();

        int[] arr = {1,2,3,4,5};
        // 인스턴스 생성과 상관 없이 인터페이스 타입으로 사용한다.
        int sum = Calc.total(arr);

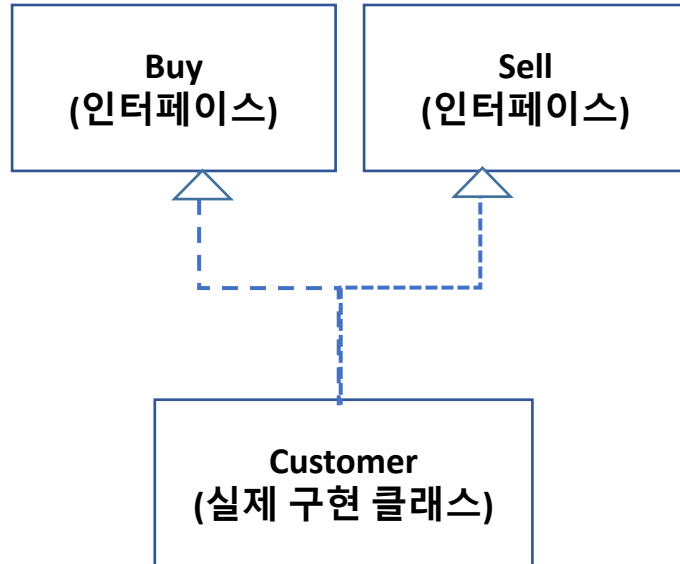
        System.out.println(sum);
    }
}
```



```
Console x
<terminated> CaccluatorTest [Java Application] C:\Users\WAdm
완벽한 계산기입니다.
myStaticMethod 메서드 입니다.
15
```



➤ 두 개의 인터페이스 구현하기



```
package interfaceTest;

public interface Buy {
    void buy();
    default void order() {
        System.out.println("구매주문");
    }
}
```

```
package interfaceTest;

public interface Sell {
    void sell();
    default void order() {
        System.out.println("판매주문");
    }
}
```



- 인터페이스(Interface)는 동시에 두개 이상 implements 가 가능하다.
- Customer 클래스는 Buy와 Sell인터페이스를 모두 구현 함.

```
public class Customer implements Buy, Sell {
```

```
@Override
```

```
public void sell() {  
    System.out.println("판매하기");  
}
```

```
@Override
```

```
public void buy() {  
    System.out.println("구매하기");  
}
```

```
// 같은 이름의 default 메서드는 재정의 하여 사용한다.
```

```
@Override
```

```
public void order() {  
    System.out.println("고객의 판매 주문");  
}
```



```
package interfaceTest;

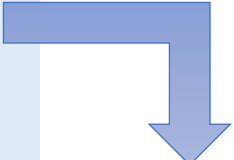
public class CustomerTest {

    public static void main(String[] args) {

        Customer customer = new Customer();

        Buy buyer = customer;
        buyer.buy();
        Sell seller = customer;
        seller.sell();

        // 가상 메서드로 order()메서드를 재정의한 customer 클래스의 order()메서드가 출력된다.
        customer.order();
        buyer.order();
        seller.order();
    }
}
```

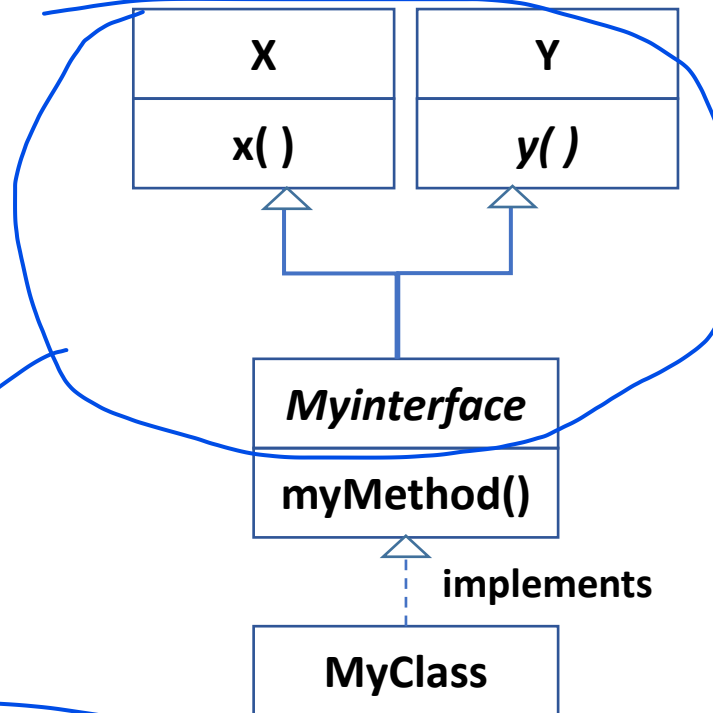


Console X
<terminated> CustomerTest (1) [Java]
구매하기
판매하기
고객의 판매 주문
고객의 판매 주문
고객의 판매 주문



➤ 인터페이스 상속

- 인터페이스 간에도 상속이 가능
- 구현 코드의 상속이 아니므로 **형 상속(type inheritance)** 라고 함



```
package interfaceTest;  
  
public interface MyInterface extends X,Y {  
    void myMethod();  
}
```

인터페이스 여러 개를
상속 받을 수 있음



```
package interfaceTest;
```

```
public interface X {  
    void x();  
}
```

```
package interfaceTest;
```

```
public interface Y {  
    void y();  
}
```

➤ x와 y 인터페이스를 MyInterface 가 상속을 받았다.

```
package interfaceTest;
```

```
public interface MyInterface extends X,Y {  
  
    void myMethod();  
}
```



```
package interfaceTest;

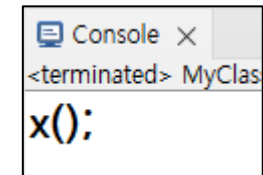
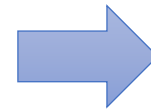
public class MyClass implements MyInterface {

    @Override
    public void x() {
        System.out.println("x():");
    }

    @Override
    public void y() {
        System.out.println("y():");
    }

    @Override
    public void myMethod() {
        System.out.println("mymethod():");
    }

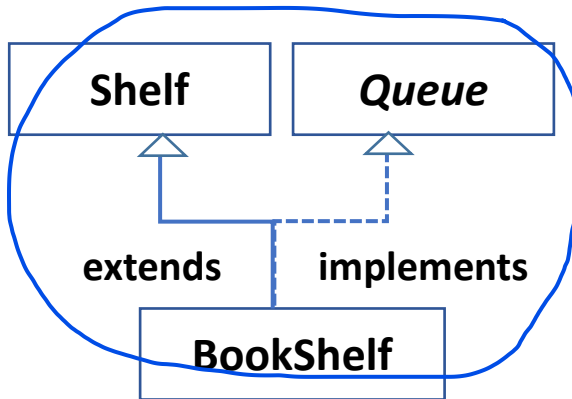
    public static void main(String[] args) {
        MyClass myclass = new MyClass();
        X xclass = myclass;
        xclass.x();
    }
}
```





➤ 인터페이스 구현과 클래스 상속 함께 사용하기

- 실제 프레임 워크(스프링, 안드로이드)를 사용하면 클래스를 상속 받고 여러 인터페이스를 구현하는 경우가 종종 있다.



```
package bookshelf;
```

```
public class BookShelf extends Shelf implements Queue {
```

```
@Override
```

```
public void inQueue(String title) {
    shelf.add(title); //ArrayList에서 add는 맨 뒤에서 부터 추가 된다.
}
```

배열에 요소 추가

```
@Override
```

```
public String deQueue() {
    return shelf.remove(0); // 제일 첫번째 자료를 삭제
}
```

맨 처음 요소를 배열에서 삭제하고 반환

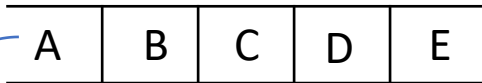
```
@Override
```

```
public int getSize() {
    return getCount();
}
```

배열에 요소 갯수 반환

- **Queue** 는 **FIFO(First In First Out)** 으로 먼저 입력된 데이터가 먼저 출력된다는 자료구조이다.

FI(First In)



FO(First Out)



```
package bookshelf;
```

```
public interface Queue {  
    void inQueue(String title); //맨 뒤로 집어 넣기  
    String deQueue(); // 꺼내기, FIFO  
    int getSize();  
}
```

Queue 인터페이스를 작성

Shelf 클래스를 작성

```
package bookshelf;
```

```
import java.util.ArrayList;
```

```
public class Shelf {  
    // 멤버변수  
    protected ArrayList<String> shelf;
```

```
    //생성자  
    public Shelf() {  
        shelf = new ArrayList<String>();  
    }  
    public ArrayList<String> getShelf() {  
        return shelf;  
    }  
    public int getCount() {  
        return shelf.size();  
    }  
}
```



```
package bookshelf;

public class BookShelf extends Shelf implements Queue {

    @Override
    public void inQueue(String title) {
        shelf.add(title); //ArrayList에서 add는 맨 뒤에서 부터 추가 된다.
    }

    @Override
    public String deQueue() {
        return shelf.remove(0); // 제일 첫번째 자료를 삭제
    }

    @Override
    public int getSize() {
        return getCount();
    }

}
```

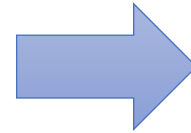


```
package bookshelf;

public class BookShelfTest {

    public static void main(String[] args) {
        Queue shelfQueue = new BookShelf();
        shelfQueue.inQueue("태백산맥1");
        shelfQueue.inQueue("태백산맥2");
        shelfQueue.inQueue("태백산맥3");

        System.out.println(shelfQueue.dequeue());
        System.out.println(shelfQueue.dequeue());
        System.out.println(shelfQueue.dequeue());
    }
}
```



Console X
<terminated> BookShe
태백산맥1
태백산맥2
태백산맥3

연습문제 -인터페이스

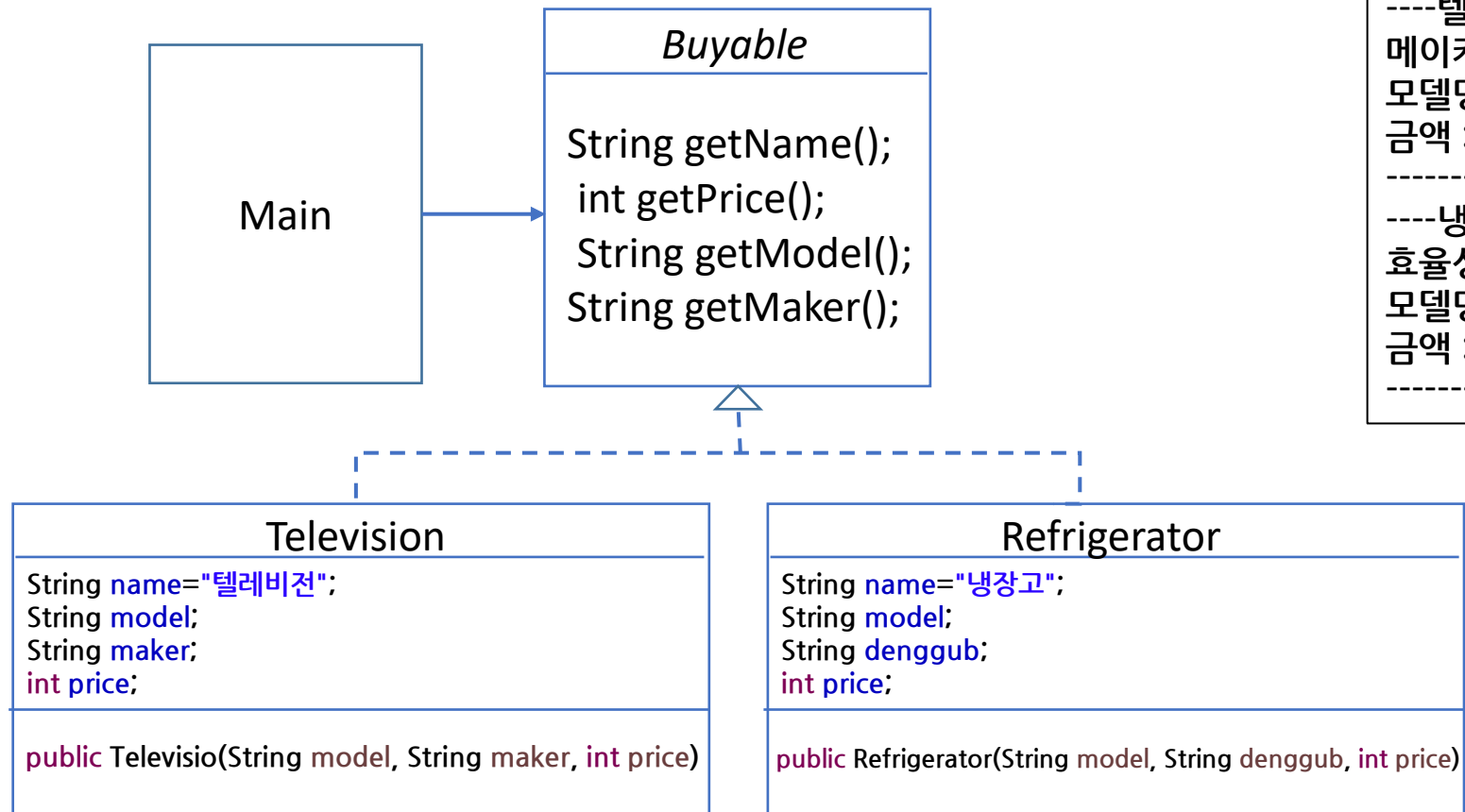




Package 명 interfaceExample01로 작성 할 것

Class 명 : Buyable, Television, Refrigerator, Main

문제1> 아래와 같이 출력되도록 작성 하시오.



<출력 결과물>

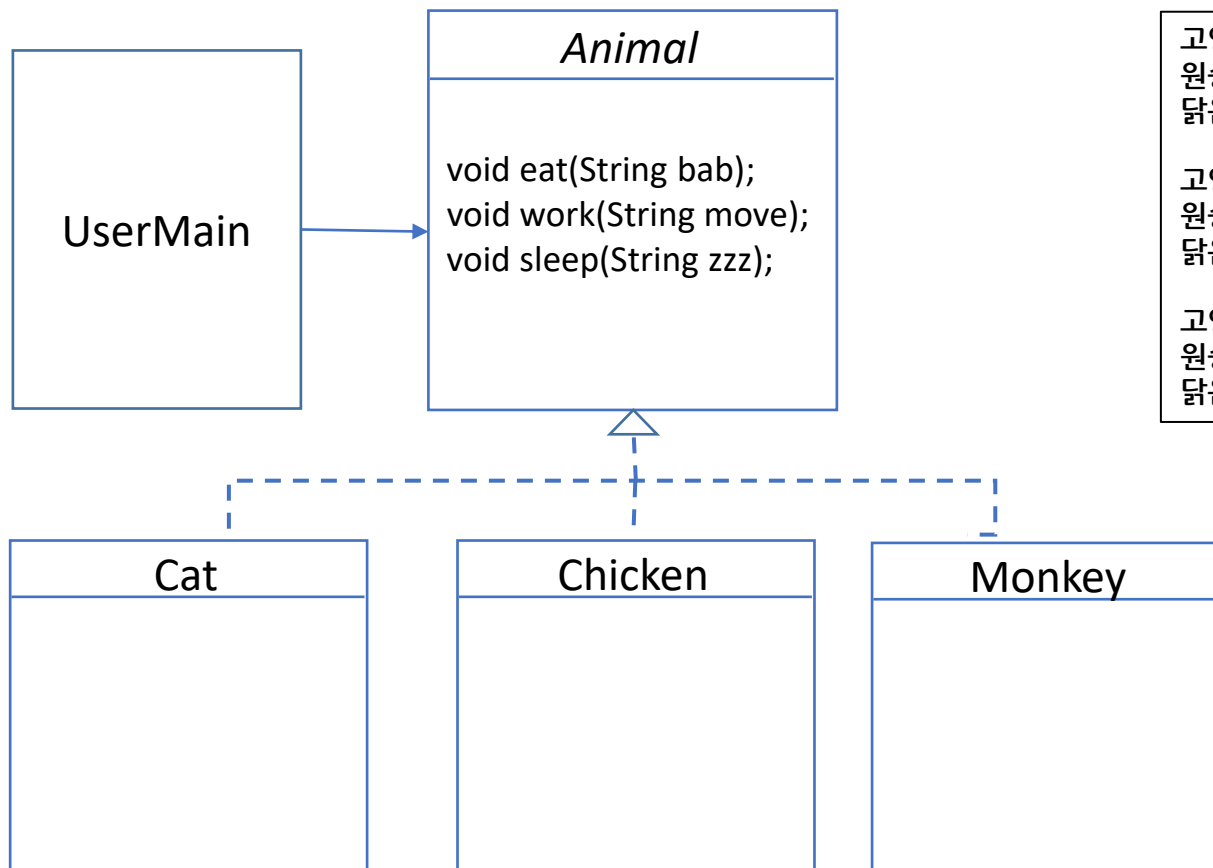
```
-----텔레비전정보 -----
메이커 :TV-samsung
모델명 : MV-2020
금액 :509,000
-----
-----냉장고정보 -----
효율성 등급 :1등급
모델명 : MV-2020
금액 : 2,000,000
-----
```



Package 명 interfaceExample02 로 작성 할 것

Class 명 : Animal, Cat, Chicken, Monkey, UserMain

문제1> 아래와 같이 출력되도록 작성 하시오.



<출력 결과물>

고양이는 생선을 먹어요.
원숭이는 과일을 먹어요.
닭은 사료를 먹어요.

고양이는 네발로걸어요.
원숭이는 네발 또는 두발로걸어요.
닭은 두발로걸어요.

고양이는 엎드려서자요.
원숭이는 누워서자요.
닭은 서서자요.



Package 명 interfaceExample03 로 작성 할 것

Class 명 : 클래스명 또는 인터페이스 이름은 알아서 작성 할 것

문제> 다음은 도형을 표현하는 인터페이스 Shape를 이용하여 원(Circle)을 구현하는 프로그램이다.

아래 조건을 만족하도록 인터페이스와 클래스를 작성하고, 실행 결과가 주어진 출력과 같도록 전체 프로그램을 완성하시오.

<조건>

1. Shape 인터페이스를 작성하시오.

- void draw();
- double getArea();
- default void redraw() 메소드를 포함하며,
- "--- 다시 그립니다."를 출력한 후 draw()를 호출하도록 구현한다.

2. Circle 클래스는 Shape 인터페이스를 구현한다.

- 반지름(radius)을 멤버 변수로 가진다.
- 생성자를 통해 반지름을 초기화한다.
- draw() 메소드는
- "반지름이 10인 원입니다." 와 같이 출력한다.
- getArea() 메소드는 원의 면적을 계산하여 반환한다.
- (원주율은 3.14를 사용한다.)

3. main() 메소드가 포함된 클래스를 작성한다.

<출력 결과>

```
Outline Task List Console X
<terminated> Main (4) [Java Application] C:\WProg
--- 다시 그립니다.
반지름이 10인 원입니다.
면적은 314.0
```